BigCodeBench/877 Evaluations

Task Description

Perform PCA on a DataFrame with:

- Scale data using StandardScaler
- Apply PCA to reduce dimensions to n_components
- Raise ValueError for: non-DataFrame input, non-numeric data, empty data, n_components > columns
- Return DataFrame with principal components

Model 1 Evaluation

model1_efficiency

Total steps: 13

- Step 1: Define function - **Essential**
- Step 2: Validate DataFrame type - **Essential**
- Step 3: Check if empty - **Essential**
- Step 4: Check numeric data - **Essential**
- Step 5: Validate n_components type - **Non-essential** (not required by spec)
- Step 6: Check n_components > 0 - **Non-essential** (not required by spec)
- Step 7: Check n_components > columns - **Essential**
- Step 8: Instantiate StandardScaler - **Essential**
- Step 9: Fit and transform scaler - **Essential**
- Step 10: Instantiate PCA - **Essential**
- Step 11: Fit and transform PCA - **Essential**
- Step 12: Convert to DataFrame - **Essential**
- Step 13: Return result - **Essential**

model1_logic

Step Coherence:

- All steps follow logically from requirements
- Additional validation steps (5, 6) are reasonable but not specified
- Clear progression from validation to implementation

Internal Consistency: No contradictions, thorough approach

model1_completeness

Problem Coverage:

- DataFrame validation: Yes
- Empty check: Yes
- Numeric data check: Yes
- n_components validation: Yes (plus extras)
- StandardScaler: Yes
- PCA implementation: Yes
- Return DataFrame: Yes

Factual Correctness: All correct

Constraint and edge case handling: Very thorough, includes additional validations

---

Model 2 Evaluation

model2_efficiency

Total steps: 7

- Step 1: Validate DataFrame - **Essential**
- Step 2: Check empty - **Essential**
- Step 3: Check numeric data - **Essential**
- Step 4: Validate n_components - **Essential**
- Step 5: Scale data - **Essential**
- Step 6: Apply PCA - **Essential**
- Step 7: Return DataFrame - **Essential**

model2_logic

Step Coherence:

- Step 1-4: Validation phase
- Step 5-7: Implementation phase
- Clear logical flow

Internal Consistency: No contradictions, concise approach

model2_completeness

Problem Coverage:

- DataFrame validation: Yes
- Empty check: Yes
- Numeric data check: Yes
- n_components validation: Yes
- StandardScaler: Yes
- PCA implementation: Yes
- Return DataFrame: Yes

Factual Correctness: All correct, mentions is_numeric_dtype

Constraint and edge case handling: Covers all required cases

Model 3 Evaluation

model3_efficiency

Total steps: 8

- Step 1: Understand requirements - **Essential**
- Step 2: Design implementation - **Essential** (planning)
- Step 3: Detail validation - **Essential**
- Step 4: Scale data - **Essential**
- Step 5: Apply PCA - **Essential**
- Step 6: Convert to DataFrame - **Essential**
- Step 7: Combine steps - **Non-essential** (meta-step)
- Step 8: Review code - **Non-essential** (meta-step)

model3_logic

Step Coherence:

- Good planning approach
- Includes meta-steps about implementation process
- Notes additional n_components > 0 check for robustness

Internal Consistency: Consistent, includes planning steps

model3_completeness

Problem Coverage:

- DataFrame validation: Yes

- Empty check: Yes
- Numeric data check: Yes (mentions select_dtypes)
- n_components validation: Yes
- StandardScaler: Yes
- PCA implementation: Yes
- Return DataFrame: Yes

Factual Correctness: Correct understanding

Constraint and edge case handling: Good coverage, suggests additional robustness

---

Model 4 Evaluation

model4_efficiency

Total steps: 5

- Step 1: Validate all inputs - **Essential**
- Step 2: Check n_components - **Essential**
- Step 3: Scale data - **Essential**
- Step 4: Apply PCA - **Essential**
- Step 5: Convert to DataFrame - **Essential**

model4_logic

Step Coherence:

- Concise grouping of validation in step 1
- Clear progression through requirements
- Notes importance of retaining original index

Internal Consistency: No contradictions, well-organized

model4_completeness

Problem Coverage:

- DataFrame validation: Yes
- Empty check: Yes
- Numeric data check: Yes
- n_components validation: Yes
- StandardScaler: Yes

- PCA implementation: Yes
- Return DataFrame: Yes

Factual Correctness: Correct, emphasizes variance sensitivity

Constraint and edge case handling: Covers all required cases

Model 5 Evaluation

model5_efficiency

Total steps: 8

- Step 1: Understand requirements - **Essential**
- Step 2: Import libraries - **Non-essential** (already in starter code)
- Step 3: Input validation start - **Essential**
- Step 4: Numeric check - **Essential**
- Step 5: n_components validation - **Essential**
- Step 6: Scale data - **Essential**
- Step 7: Apply PCA - **Essential**
- Step 8: Edge cases discussion - **Essential**

model5_logic

Step Coherence:

- Good flow from understanding to implementation
- Mentions select_dtypes for numeric checking
- Notes n_components must be at least 1

Internal Consistency: Consistent approach

model5_completeness

Problem Coverage:

- DataFrame validation: Yes
- Empty check: Yes
- Numeric data check: Yes (select_dtypes)
- n_components validation: Yes
- StandardScaler: Yes
- PCA implementation: Yes

- Return DataFrame: Yes

Factual Correctness: Correct, notes preserving original index

Constraint and edge case handling: Good discussion of edge cases

---

Model 6 Evaluation

model6_efficiency

Total steps: 4

- Step 1: Understand task - **Essential**
- Step 2: Validate inputs - **Essential**
- Step 3: Implement PCA process - **Essential**
- Step 4: Format output - **Essential**

model6_logic

Step Coherence:

- Very high-level grouping
- Each step contains multiple sub-tasks
- Clear understanding of requirements

Internal Consistency: No contradictions, high-level view

model6_completeness

Problem Coverage:

- DataFrame validation: Yes
- Empty check: Yes
- Numeric data check: Yes
- n_components validation: Yes
- StandardScaler: Yes
- PCA implementation: Yes
- Return DataFrame: Yes

Factual Correctness: Correct understanding

Constraint and edge case handling: Covers required cases

Summary

- **Most Complete**: Model 1 (includes additional validations beyond requirements)
- **Most Efficient**: Models 4 and 6 (concise with essential steps only)
- **Best Logic**: Model 4 (well-organized, concise, complete)
- **Issues**: Model 1 and 3 have non-essential steps; Model 5 mentions imports unnecessarily